

MySQL Business Case Study

Practice Version for Trainees

QuickKart Retail Order Management System | Questions only, solutions removed

Trainer note: This practice file intentionally removes only the SQL solutions. Table creation scripts and insert data scripts are retained so trainees can execute the setup and solve the questions independently.

1. Business Case Study

QuickKart is an online retail company that sells electronics, fashion, groceries, and home products. The company wants to maintain customer details, product inventory, order transactions, payments, returns, and marketing leads in a MySQL database.

The business team wants to answer questions like:

- Which customers placed orders?
- Which products are selling the most?
- What is the total revenue by category?
- Which customers are active in both customer and marketing lead lists?
- Which leads have not yet become customers?
- How can order status and stock be updated safely using transactions?
- How can incorrect orders be deleted or rolled back?

This case study covers:

- DML: INSERT, UPDATE, DELETE
- DQL: SELECT, FROM, WHERE, ORDER BY, GROUP BY
- TCL: COMMIT, ROLLBACK
- Logical operators: AND, OR, NOT
- Aliases
- Set operations: UNION, INTERSECT, EXCEPT
- Aggregate functions: COUNT, SUM, AVG, MIN, MAX

2. Database and Table Creation Script

```
DROP DATABASE IF EXISTS quickkart_db;
CREATE DATABASE quickkart_db;
USE quickkart_db;

CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    city VARCHAR(50),
    state VARCHAR(50),
    email VARCHAR(100) UNIQUE,
    customer_type VARCHAR(20),
    registered_date DATE
) ENGINE = InnoDB;

CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    category VARCHAR(50),
    unit_price DECIMAL(10,2),
    stock_quantity INT,
    status VARCHAR(20)
) ENGINE = InnoDB;

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
```

```

    customer_id INT,
    order_date DATE,
    order_status VARCHAR(30),
    payment_mode VARCHAR(30),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
) ENGINE = InnoDB;

CREATE TABLE order_items (
    item_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT,
    selling_price DECIMAL(10,2),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
) ENGINE = InnoDB;

CREATE TABLE returns (
    return_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    return_reason VARCHAR(100),
    return_date DATE,
    refund_amount DECIMAL(10,2),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
) ENGINE = InnoDB;

CREATE TABLE marketing_leads (
    lead_id INT PRIMARY KEY,
    lead_name VARCHAR(100),
    city VARCHAR(50),
    email VARCHAR(100),
    lead_source VARCHAR(50)
) ENGINE = InnoDB;

```

3. Insert Data Script

```

INSERT INTO customers
(customer_id, customer_name, city, state, email, customer_type, registered_date)
VALUES
(1, 'Arun Kumar', 'Chennai', 'Tamil Nadu', 'arun.kumar@gmail.com', 'Premium', '2024-01-10'),
(2, 'Meena Sharma', 'Bangalore', 'Karnataka', 'meena.sharma@gmail.com', 'Regular', '2024-02-15'),
(3, 'Rahul Verma', 'Hyderabad', 'Telangana', 'rahul.verma@gmail.com', 'Premium', '2024-03-05'),
(4, 'Priya Nair', 'Chennai', 'Tamil Nadu', 'priya.nair@gmail.com', 'Regular', '2024-03-22'),
(5, 'Karthik Raj', 'Mumbai', 'Maharashtra', 'karthik.raj@gmail.com', 'Premium', '2024-04-18'),
(6, 'Divya Menon', 'Kochi', 'Kerala', 'divya.menon@gmail.com', 'Regular', '2024-05-11'),
(7, 'Sanjay Gupta', 'Delhi', 'Delhi', 'sanjay.gupta@gmail.com', 'Regular', '2024-06-09'),
(8, 'Anitha Rao', 'Bangalore', 'Karnataka', 'anitha.rao@gmail.com', 'Premium', '2024-07-01');

INSERT INTO products
(product_id, product_name, category, unit_price, stock_quantity, status)
VALUES
(101, 'Laptop', 'Electronics', 55000.00, 20, 'Active'),
(102, 'Mobile Phone', 'Electronics', 25000.00, 35, 'Active'),
(103, 'Headphones', 'Electronics', 2500.00, 80, 'Active'),
(104, 'Formal Shirt', 'Fashion', 1500.00, 100, 'Active'),
(105, 'Sports Shoes', 'Fashion', 3500.00, 60, 'Active'),
(106, 'Rice Bag 25kg', 'Grocery', 1800.00, 50, 'Active'),
(107, 'Cooking Oil 5L', 'Grocery', 900.00, 75, 'Active'),
(108, 'Office Chair', 'Home', 7000.00, 15, 'Active'),
(109, 'Study Table', 'Home', 8500.00, 10, 'Inactive'),
(110, 'Smart Watch', 'Electronics', 12000.00, 25, 'Active');

INSERT INTO orders
(order_id, customer_id, order_date, order_status, payment_mode)
VALUES
(1001, 1, '2024-08-01', 'Delivered', 'UPI'),
(1002, 2, '2024-08-03', 'Delivered', 'Credit Card'),
(1003, 3, '2024-08-05', 'Shipped', 'UPI'),
(1004, 4, '2024-08-07', 'Cancelled', 'Cash on Delivery'),

```

```

(1005, 5, '2024-08-09', 'Delivered', 'Debit Card'),
(1006, 1, '2024-08-11', 'Delivered', 'UPI'),
(1007, 6, '2024-08-12', 'Processing', 'Credit Card'),
(1008, 7, '2024-08-13', 'Delivered', 'UPI'),
(1009, 8, '2024-08-14', 'Shipped', 'Debit Card'),
(1010, 3, '2024-08-15', 'Delivered', 'Credit Card');

INSERT INTO order_items
(item_id, order_id, product_id, quantity, selling_price)
VALUES
(1, 1001, 101, 1, 55000.00),
(2, 1001, 103, 2, 2500.00),
(3, 1002, 104, 3, 1500.00),
(4, 1002, 106, 2, 1800.00),
(5, 1003, 102, 1, 25000.00),
(6, 1003, 110, 1, 12000.00),
(7, 1004, 105, 1, 3500.00),
(8, 1005, 108, 1, 7000.00),
(9, 1005, 107, 4, 900.00),
(10, 1006, 102, 1, 25000.00),
(11, 1006, 103, 1, 2500.00),
(12, 1007, 106, 1, 1800.00),
(13, 1008, 105, 2, 3500.00),
(14, 1008, 104, 2, 1500.00),
(15, 1009, 110, 1, 12000.00),
(16, 1010, 101, 1, 55000.00),
(17, 1010, 107, 2, 900.00);

INSERT INTO returns
(return_id, order_id, product_id, return_reason, return_date, refund_amount)
VALUES
(1, 1002, 104, 'Size issue', '2024-08-06', 1500.00),
(2, 1005, 107, 'Damaged package', '2024-08-12', 900.00),
(3, 1008, 105, 'Wrong size', '2024-08-16', 3500.00);

INSERT INTO marketing_leads
(lead_id, lead_name, city, email, lead_source)
VALUES
(1, 'Arun Kumar', 'Chennai', 'arun.kumar@gmail.com', 'Instagram'),
(2, 'Meena Sharma', 'Bangalore', 'meena.sharma@gmail.com', 'Google Ads'),
(3, 'Naveen Reddy', 'Hyderabad', 'naveen.reddy@gmail.com', 'LinkedIn'),
(4, 'Swetha Iyer', 'Chennai', 'swetha.iyer@gmail.com', 'Referral'),
(5, 'Priya Nair', 'Chennai', 'priya.nair@gmail.com', 'Instagram'),
(6, 'Vikram Singh', 'Delhi', 'vikram.singh@gmail.com', 'Google Ads');

```

4. Business Questions for Trainee Practice

Solutions are intentionally removed in this version. Trainees should write and execute their own SQL queries using the setup scripts above.

Question 1: Display all customers from Chennai.

Write your SQL query below:

Question 2: Display product name, category, and unit price of all active products.

Write your SQL query below:

Question 3: Display all orders placed using UPI payment mode.

Write your SQL query below:

Question 4: Display products ordered by highest price first.

Write your SQL query below:

Question 5: Display all premium customers from Chennai or Bangalore.

Write your SQL query below:

Question 6: Display all products that are not inactive.

Write your SQL query below:

Question 7: Display customer order details using aliases.

Write your SQL query below:

Question 8: Find the total number of customers.

Write your SQL query below:

Question 9: Find the total stock available across all products.

Write your SQL query below:

Question 10: Find the average product price.

Write your SQL query below:

Question 11: Find the minimum and maximum product price.

Write your SQL query below:

Question 12: Find total revenue generated from each order.

Write your SQL query below:

Question 13: Find category-wise total revenue.

Write your SQL query below:

Question 14: Find category-wise number of products.

Write your SQL query below:

Question 15: Display customers whose orders are delivered and paid through UPI.

Write your SQL query below:

Question 16: Display a combined list of customer emails and marketing lead emails using UNION.

Business need: The marketing team wants one unique email list for campaign communication.

Write your SQL query below:

Question 17: Find people who are both existing customers and marketing leads using INTERSECT.

Business need: The marketing team wants to identify leads who already converted into customers.

Write your SQL query below:

Question 18: Find marketing leads who are not yet customers using EXCEPT.

Business need: The sales team wants to focus on leads who have not converted yet.

Trainer hint: In MySQL, use EXCEPT for set difference. Oracle uses MINUS.

Write your SQL query below:

Question 19: Update stock quantity after an order is delivered and commit the transaction.

Business need: When order 1001 is delivered, reduce stock for the ordered products.

Write your SQL query below:

Question 20: Delete a cancelled order safely using ROLLBACK.

Business need: The operations team tries to delete cancelled order 1004, but later realizes it should not be deleted. So rollback is used.

Write your SQL query below:

5. Additional DML Practice Tasks

A. Insert a new customer

Write your SQL query below:

B. Update a customer type from Regular to Premium

Write your SQL query below:

C. Delete an inactive product

Write your SQL query below:

6. Topic Coverage Mapping

Topic	Covered In Questions
INSERT	Insert scripts, Additional Practice A
UPDATE	Question 19, Additional Practice B
DELETE	Question 20, Additional Practice C
SELECT	Questions 1 to 18
FROM	Questions 1 to 18
WHERE	Questions 1, 2, 3, 5, 6, 15, 19, 20
ORDER BY	Question 4
GROUP BY	Questions 12, 13, 14
COMMIT	Question 19
ROLLBACK	Question 20
AND	Questions 5, 15, Additional Practice C
OR	Question 5
NOT	Question 6
Aliases	Question 7
UNION	Question 16
INTERSECT	Question 17
MINUS / EXCEPT	Question 18
COUNT	Questions 8, 14
SUM	Questions 9, 12, 13
AVG	Question 10
MIN	Question 11
MAX	Question 11

7. Trainer Notes

- Use this file as the trainee practice version.
- Ask learners to first execute the table scripts, then insert scripts, then solve the questions independently.

- Keep the solved version separately as the answer key for trainer validation.
- For transactions, ask learners to use START TRANSACTION, COMMIT, and ROLLBACK in a controlled practice environment.
- For set operations, remind learners that MySQL uses EXCEPT while Oracle commonly uses MINUS.